

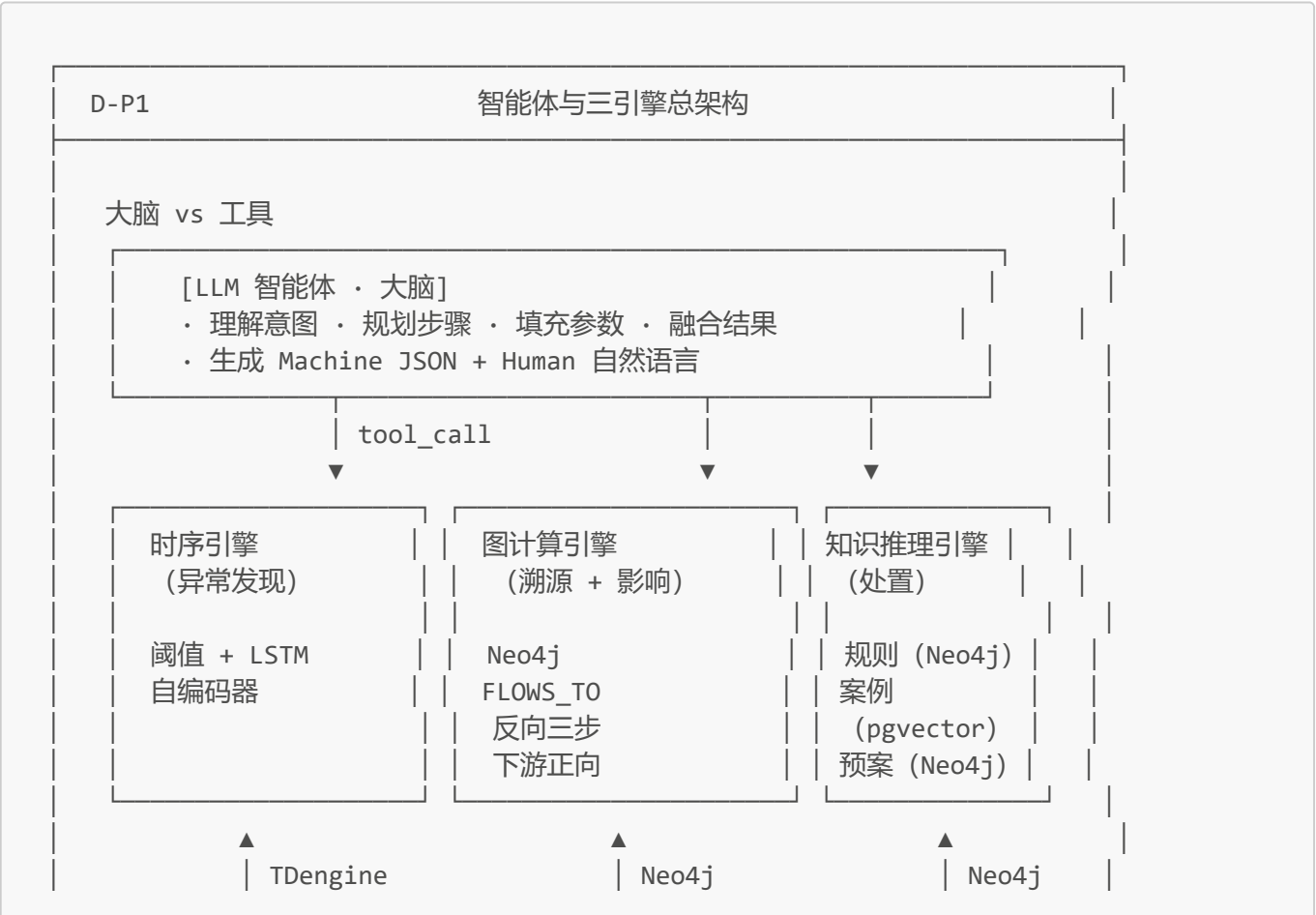
Part D · AI 智能分析 (5 页)

**定位：**讲三大引擎（时序 / 图计算 / 知识推理）原理、智能体编排机制、算法细节。不含实操代码，但**网络结构、算法推理步骤、数据模型**要讲清楚。部分页大量复用 **技术部分PPT.md**。

页面总览

| 页码   | 标题                       | 一句话                           | 复用来源               |
|------|--------------------------|-------------------------------|--------------------|
| D-P1 | 智能体与三引擎总架构               | 大脑 vs 工具 + 主动/被动模式 + 闭环       | P1.6               |
| D-P2 | 时序引擎：阈值 + LSTM 自编码器      | 阈值毫秒筛 + LSTM 重建误差判异常          | P2.2 / P2.2.1      |
| D-P3 | 图计算引擎：溯源三步 + 下游影响        | Neo4j 反向三步 + 四级预警             | P3.2 / P3.3 / P3.4 |
| D-P4 | 知识推理引擎：规则 + 案例 + 预案      | 48 规则 + 200+ 案例（16 维） + 12 预案 | P4.2 / P4.3 / P4.4 |
| D-P5 | 智能体编排：双输出 + Tool Calling | LLM 调度三引擎 + Machine/Human 双输出 | P2.3               |

D-P1 · 智能体与三引擎总架构





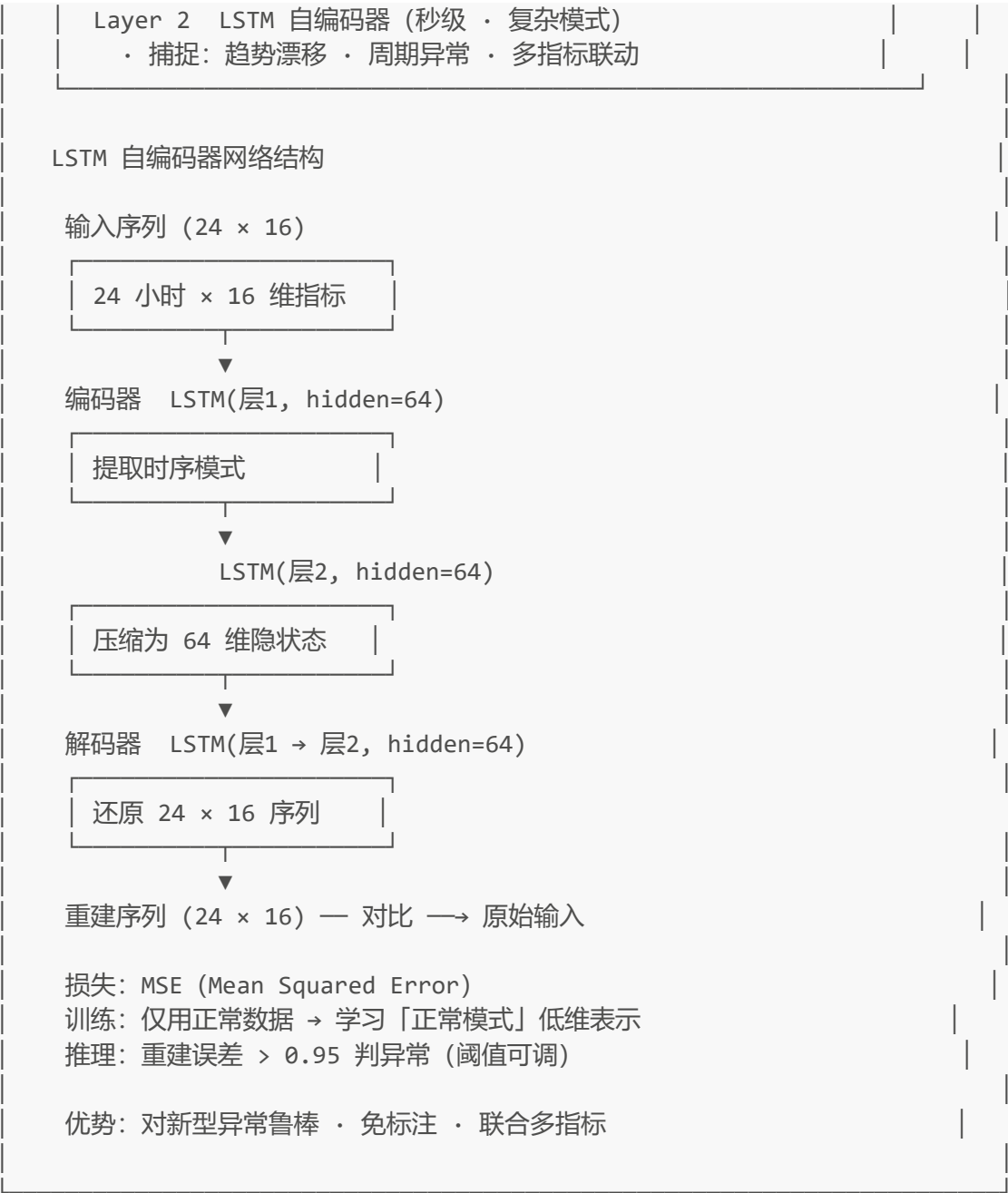
讲稿要点

- **大脑 vs 工具分工**：LLM 作为智能体负责**理解 / 规划 / 生成**，三大引擎作为**专业计算工具**——「大脑不搞数值计算，工具不做自然语言生成」
- **三大引擎分工**：
  - **时序引擎**（异常发现）：阈值 + LSTM 自编码器混合策略，消费 TDengine 时序数据
  - **图计算引擎**（溯源 + 影响）：Neo4j 流域拓扑 + Cypher 双向遍历
  - **知识推理引擎**（处置）：规则（Neo4j）+ 案例（pgvector）+ 预案（Neo4j）三层
- **两种工作模式**：
  - **主动模式**：每 5 分钟定时巡检全站，发现异常即自动串起「发现 → 溯源 → 处置」完整闭环
  - **被动模式**：用户在前端 AI 对话入口提问，Agent 判意图 → 选工具组合 → 融合回答
- **三场景闭环**：异常发现、污染溯源、处置决策——对应三大引擎各自核心任务
- **技术事实**：智能体与引擎的解耦让 LLM 换型（GPT / DeepSeek / Qwen 等）不影响引擎实现，引擎升级也不影响对话接口

下一页过渡：三大引擎先看第一个——时序引擎是怎么把异常从千万个时序点里捞出来的。

D-P2 · 时序引擎：阈值 + LSTM 自编码器混合策略





讲稿要点

- **为什么混合**: 阈值简单、快、毫秒级筛超标; LSTM 复杂、慢、捕捉**阈值杀不出来的复杂模式** (趋势漂移、周期异常、多指标联动); 两者串行互补
- **Layer 1 阈值层**: 基础参考 **GB 3838-2002 地表水环境质量标准**, 允许站点级自定义阈值覆盖; 输出按四级预警 (critical / high / medium / low)
- **Layer 2 LSTM 自编码器网络结构**:
  - **输入**: 24 小时 × 16 维指标序列 (11 项水质字段 + 5 项衍生 / 环境特征)
  - **编码器**: LSTM **2 层**、**隐藏维度 64**, 逐步把时序压缩为 64 维隐状态
  - **解码器**: LSTM **2 层**, 把隐状态还原为 24 × 16 序列
- **损失函数**: 重建误差 MSE (Mean Squared Error)
- **训练策略**: **只用正常数据**训练, 让网络学习「什么是正常模式」的低维表示; **不需要异常标注**——这是自编码器用于异常检测的核心好处
- **推理判定**: 把当前窗口喂进网络, 重建误差 > **0.95** 判异常 (阈值可按站点调节)

- **优势小结：**对训练集里没见过的新型异常鲁棒、免标注冷启动、能捕捉多指标联动（比如 DO 骤降 + 氨氮上升的典型污染信号组合）
- **衔接设计：**阈值触发 → 直接走图计算溯源；LSTM 触发 → 先人工复核再决定是否溯源（避免误报扩散）

**下一页过渡：**异常被捞出来后，下一步问的是「它从哪里来」。

D-P3 · 图计算引擎：溯源三步 + 下游影响

D-P3图计算引擎：溯源三步 + 下游影响

流域拓扑建模 (Neo4j)

(Station:上游) -[FLOWS\_TO {distance, travel\_time}]-> (Station:下游)

↓ HAS

(PollutionSource) -HAS\_TYPE- (PollutionType)

HAS\_RULE → (Rule)

HAS\_PLAN → (EmergencyPlan)

反向溯源三步推理 (异常站 → 可疑污染源)

步骤 1 · 拓扑反向

MATCH (up)-[:FLOWS\_TO\*1..N]->(target)

→ 上游候选站集合

步骤 2 · 时间相关性

Δt 实际 = target.anomaly\_t - up.anomaly\_t

Δt 理论 = FLOWS\_TO.travel\_time

吻合度 = 1 - |Δt 实际 - Δt 理论| / Δt 理论

步骤 3 · 空间 + 特征匹配

上游候选周边 PollutionSource 的 PollutionType 特征

与目标站异常指标组合做匹配

综合置信度公式

confidence = w1 × 距离衰减权重

+ w2 × 时间相关性吻合度

+ w3 × 特征匹配度

下游影响正向遍历

MATCH (src)-[f:FLOWS\_TO\*1..M]->(down)

预计到达时间 ETA = Σ f.travel\_time

分级策略：

· 饮用水源站 → critical 启动应急预案

· 普通断面 → high 加密监测

- 农业灌溉

→ medium

通知用户

· 其他

→ low

记录观察

讲稿要点

- **流域拓扑建模**: Neo4j 把站点与排污源织成一张图——Station 之间用 FLOWS\_TO 关系携带 distance 和 travel\_time 两个属性，精确描述水流方向与传输时间
- **反向溯源三步推理**:
  - **步骤 1 · 拓扑反向**: 用 Cypher MATCH (up)-[:FLOWS\_TO\*1..N]->(target) 找出所有能流到异常站的上游候选集；N 是可达跳数上限
  - **步骤 2 · 时间相关性**: 上下游异常的**实际时间差**与 FLOWS\_TO.travel\_time 的**理论传输时间**做比对，吻合度越高越可疑
  - **步骤 3 · 空间 + 特征匹配**: 上游候选站周边的 PollutionSource 的排放特征（通过 PollutionType）与目标站的异常指标组合做匹配，锁定最可能的排污源
- **综合置信度公式**:  $w1 \cdot \text{距离衰减} + w2 \cdot \text{时间相关性} + w3 \cdot \text{特征匹配度}$ ——三维度加权得到每个候选源的置信度打分
- **下游影响正向遍历**: 方向反过来——以污染源为起点向下游扩散，累加 FLOWS\_TO.travel\_time 算出每个下游站的**预计到达时间 ETA**
- **四级预警分级策略**: 下游站**按站点类型**触发不同响应
  - 饮用水源站 → critical，启动应急预案
  - 普通断面 → high，加密监测频次
  - 农业灌溉 → medium，通知用户
  - 其他 → low，记录观察
- **技术事实**: 图计算的三步推理是**可视化三步骤**，前端在溯源页面用不同透明度展示候选集 → 吻合度 → 最终排名，让调度员看到推理过程而不是只看结论

**下一页过渡**: 找到源头之后，下一个问题是「怎么处置」——把结论交给知识推理引擎。

D-P4 · 知识推理引擎：规则 + 案例 + 预案三层



Layer 1 · 规则引擎 (确定性)

污染类型 → 规则 (指标 / 条件 / 权重)

$\text{score}(\text{type}) = \sum \text{命中条件} \times \text{条件权重}$

启动时加载 Neo4j 规则到内存缓存 · 支持热重载

输出: 污染类型 Top-3 + 得分

Layer 2 · 案例推理 (相似性)

16 维特征向量 =

- 11 项水质指标值
- 关键比值 (N/P · BOD/COD)
- 环境因子 (季节 · 时刻 · 站点类型 · 流域 · 天气)

pgvector 余弦距离 + Top-K

IVFFlat 索引参数: lists / probes 精度与性能权衡

输出: Top-K 相似案例 + 当年处置结果

Layer 3 · 预案匹配 (可执行)

MATCH (type)-[:HAS\_PLAN]->(plan)-[:INVOLVES]->(dept)

预案字段: 处置步骤 · 优先级 · 时限 · 责任部门

多部门协同决策 (INVOLVES 关系图)

输出: 预案清单 + 部门路由

三层融合输出

污染定性 (规则 Top-1) + 相似案例 (向量 Top-3) + 预案清单 (图匹配) |

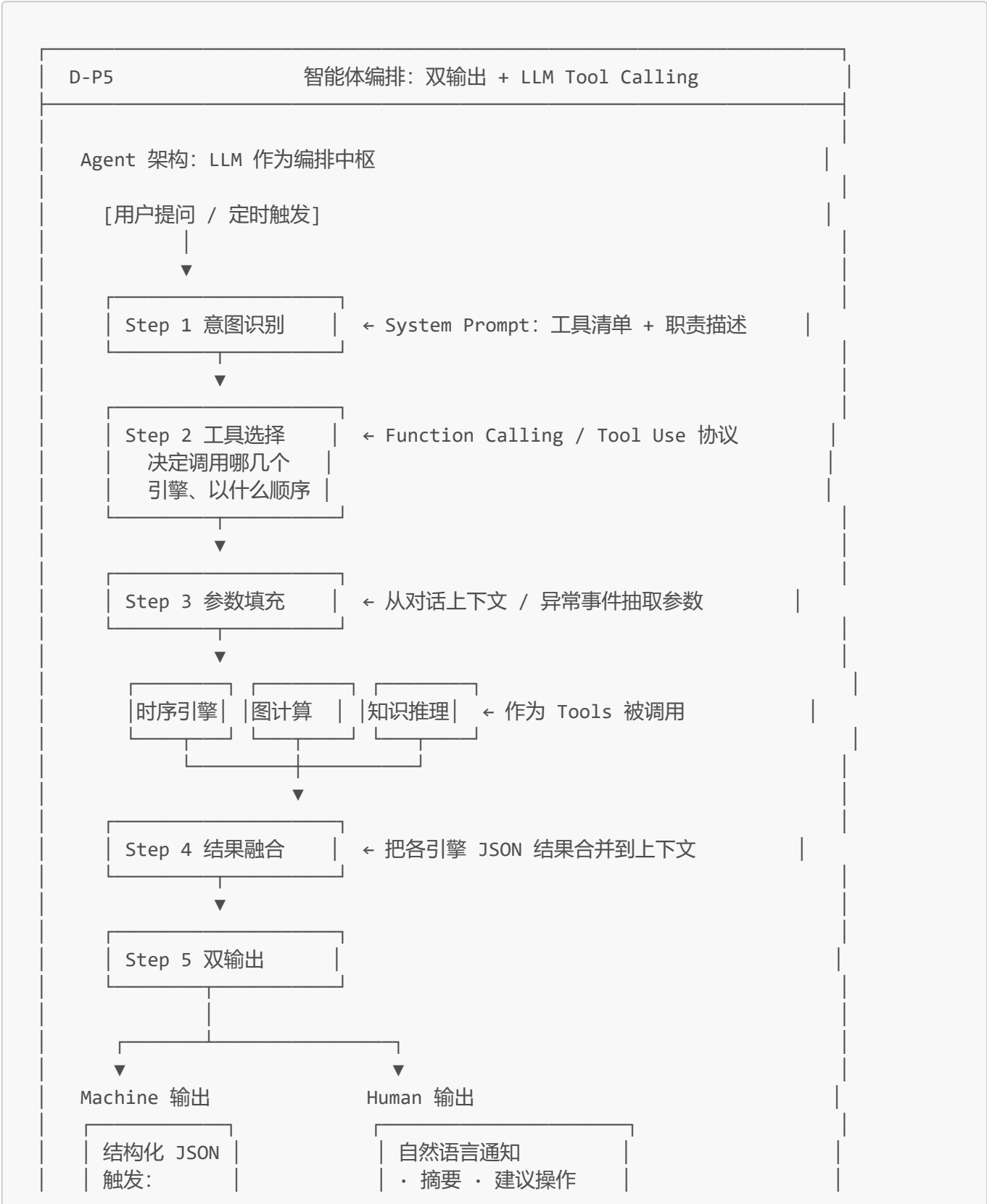
## 讲稿要点

- **双数据源设计**: 规则性知识 (Neo4j) 与经验性知识 (pgvector) 互补
  - 规则性: 7 类污染、**48 条规则**、12 类预案、6 个部门, **确定性推理**
  - 经验性: **200+ 历史案例**, 每个案例 **16 维特征向量**, **相似性借鉴**
- **Layer 1 规则引擎**:
  - 每条规则绑定到污染类型, 包含「指标 + 条件 + 权重」
  - 打分公式:  $\text{score}(\text{type}) = \sum \text{命中条件} \times \text{权重}$
  - **启动时一次性加载 Neo4j 规则到内存缓存** (每次推理不查图), 支持热重载
  - 输出污染类型 Top-3 + 得分
- **Layer 2 案例推理**:
  - **16 维特征向量构造**: 11 项水质指标值 + 关键比值 (N/P、BOD/COD) + 环境因子 (季节 / 时刻 / 站点类型 / 流域 / 天气)
  - pgvector 余弦距离 + Top-K; **IVFFlat 索引参数** (lists 决定分桶数、probes 决定查询访问桶数) 在精度与性能之间做权衡
  - 输出 Top-K 相似案例 + 当年实际处置结果
- **Layer 3 预案匹配**:
  - Cypher 匹配: `MATCH (type)-[:HAS_PLAN]->(plan)-[:INVOLVES]->(dept)`
  - 预案字段: 处置步骤 / 优先级 / 时限 / 责任部门
  - **多部门协同**: 通过 **INVOLVES** 关系图驱动跨部门路由决策

- **三层融合输出：**污染定性（规则 Top-1）+ 相似案例（向量 Top-3）+ 预案清单（图匹配）——同时满足「定性 + 佐证 + 可执行」三个维度
- **技术事实：**这个设计的本质是**把可解释的规则（Neo4j）和不可解释的相似性（向量）分开存、分开查**，既保持知识工程可审计，又获得相似案例的快速借鉴

**下一页过渡：**三大引擎各司其职，最后看智能体怎么把它们编排起来、输出给下游。

D-P5 · 智能体编排：双输出 + LLM Tool Calling



· 告警推送  
· 工单派发  
· 报告生成

· 下一步按钮选项  
LLM 基于 Prompt 模板  
变量插槽 + 格式约束

双输出一致性：同一次推理上下文，两套 Prompt，保持事实对齐  
可追溯：所有工具调用 + 参数 + 结果写入对话日志，支持复盘

## 讲稿要点

- **Agent 架构**：LLM 作为编排中枢，三大引擎作为 Tools；整个推理过程遵循「意图识别 → 工具选择 → 参数填充 → 结果融合 → 双输出」五步
- **Function Calling / Tool Use 协议**：每个引擎暴露为一个 Tool，带有名称、描述、参数 schema；LLM 根据系统 Prompt 里的工具清单自主决定调用哪几个、什么顺序
- **多轮对话上下文**：工具调用结果以消息形式回填到对话上下文，支持多轮追问；同一条异常事件的多次追问复用工具结果缓存
- **双输出设计**：同一次推理生成两套输出
  - **Machine 输出**：结构化 JSON，供下游系统消费——触发告警推送、工单派发、报告生成
  - **Human 输出**：LLM 基于 Prompt 模板生成自然语言——摘要 + 建议操作 + 下一步按钮选项，直接投递给值班员
- **Prompt 模板设计**：
  - **变量插槽**：把异常事件、溯源置信度、预案清单等结构化结果插入模板
  - **格式约束**：要求 LLM 输出分段、限字数、使用统一术语，避免每次回答风格漂移
  - **双输出一致性**：Machine 与 Human 用同一次推理的上下文、两套 Prompt 分别生成，保证两套输出的事实对齐、不会出现「JSON 说是氨氮，文字说是 COD」的矛盾
- **被动模式下的查询流程**：用户提问 → Agent 判意图 → 选择引擎组合（比如「A 站上周异常是什么原因」→ 时序查历史 + 图计算查上游 + 知识推理给定性）→ 综合回答
- **可追溯**：所有工具调用的名称、参数、返回值都写进对话日志，支持事后复盘与专家审核——这是知识工程级产品的必备要求

**结尾收束**：「三大引擎 + 智能体编排，把数据驱动升级成决策驱动。异常被时序引擎发现、被图计算引擎追溯、被知识推理引擎匹配预案，再由智能体统一输出给下游系统和现场人员——这就是四栈之上，水环境平台的完整技术闭环。」